

The Federated Harbor:

Identity, Coordination, and Settlement Across Administrative Domains

Erich Owens

Curiositech LLC

engineering@portdaddy.dev

August 2026

Version 1.0 (revised pre-print)

Abstract

A demonstration is scheduled for 4pm. Alice runs the back end on her laptop; Bob runs the front end on his. Each operator has a working local harbor, but Alice's `db:write` card is meaningless to Bob's daemon, Bob cannot yet observe Alice's revocations, and a bond posted at one harbor cannot automatically settle damage at the other. This paper specifies the missing federation boundary: a cross-harbor capability-transfer ceremony with composed attenuation, revocation dissemination through a witnessed anti-entropy overlay, a conditional escrow construction, and bonded admission for new harbors. The assurance case is deliberately narrower than earlier drafts. Epidemic dissemination supplies an expected $\Theta(\log m)$ result only under a connected reliable-round model, not a hard freshness deadline. A signed escrow promise makes theft detectable but does not prevent it; the extractable-value bound requires a custody ledger that structurally exposes only recipient-whitelisted transitions. Cross-harbor conservation is stated as a bucket-partition invariant and model-checked only at the recorded finite bound. The protocol models and federation runtime remain partial. The contribution is therefore a falsifiable design and research boundary, not a claim that federation is already deployed or trustless.

Keywords: federated identity, capability delegation, cross-realm authorization, mechanism design, distributed trust, certificate transparency, atomic settlement, mutual suspicion.

Reading time: approximately 45 minutes end-to-end. Chapters V and VI, The Anchor Protocol (Owens 2026) and The Bonded Commons (Owens 2026), supply the cryptographic and economic primitives this paper extends. Either of those can be read first or in parallel; this paper is the federation half. The Reader's Map below provides shorter routes for specific audiences.

PORT DADDY COORDINATION PAPERS

CHAPTER VII OF VII

V. The Anchor Protocol. Authentication, delegation, attenuation, and revocation inside one harbor.

VI. The Bonded Commons. Evidence, bonding, advisory claims, and the incentive conditions for cooperation.

VII. The Federated Harbor (this chapter). Identity, revocation, and settlement across mutually sovereign machines.

Chapters I–IV form the product arc; Chapters V–VII provide the protocol, economic, and federation deep dives. Every chapter is independently readable.

Reader's Map

The paper has a topology: it opens with the gestalt (§1–§2), then moves through three load-bearing primitives in order of cryptographic depth (capability transfer, revocation, settlement), then steps back to admission, failure modes, and limitations. The figures are designed to be readable in sequence without the surrounding prose; if you want the five-figure summary, that path is given below.

Layer	Load-bearing artifact	Lives at
(1) Topology	Two harbors, witness log, settlement escrow as a four-element gestalt	§2; Figure 1
(2) Cap. transfer	Four-message ceremony; ProVerif model of cross-realm authenticity & attenuation, <i>conditional on a witness-log freshness assumption</i> (§4.5)	§4; Figure 3
(3) Revocation	Conditional epidemic-dissemination expectation; partition caveat	§5; Figure 4
(4) Settlement	Recipient-whitelisted custody condition; bucket-partition Conservation	§6; Figure 5
(5) Threat bands	What is mechanized, what is bounded, what is open	§12; Figure 2

Suggested reading paths by reader type:

- **Beginner / new to the series.** Read §1 (the two-machine problem) and §8 (Alice and Bob, end-to-end). The worked example is written to be read cold. Look at the five figures in order. Skip the proofs on the first pass.
- **Distributed-systems engineer.** §2 (the topology), §4 (transfer ceremony), §5 (gossip and equivocation). Chapter V (*The Anchor Protocol*) for the cryptographic substrate. Then §12 for the honest open questions.
- **Formal-methods reviewer.** Jump to §9 (the TLA+ and ProVerif extensions) and Appendix A (the verification status table). Threat bands in Figure 2 are the most concentrated signal of what is and is not mechanized.
- **Cryptoeconomic / mechanism-design reader.** §6 (bounded escrow) and §7 (cold-start admission via bonded sponsorship). The proposal's equilibrium contributions are stated here

as open work; that is honest, not coy.

- **Security-engineering reviewer.** §3 (threat model) first, then the three primitive sections in order. §10 for the operationally interesting cases the protocols do not close.

Volume Context. Chapter V (*The Anchor Protocol*) formalizes process identity on a single machine. Chapter VI (*The Bonded Commons*) establishes pre-transactional commons authority and collateralized incentive alignment. This chapter extends both into the cross-machine, cross-organization regime. Specific primitives are cited directly (e.g., Chapter V §2.4, Chapter VI §2) rather than re-derived.

Contents

1	Introduction: The Two-Machine Problem	6
1.1	What this paper is and is not	6
1.2	Contributions	7
2	The Federated Authority	8
2.1	Harbor sovereignty as a property characterization	8
2.2	Inter-harbor scope	9
2.3	The witness log	9
2.4	What the federation is not	10
3	Threat Model	10
3.1	In-scope attacks	10
3.2	Out-of-scope attacks	11
3.3	Threat-band visualization	11
4	Cross-Harbor Capability Transfer	12
4.1	The four-message ceremony	12
4.2	Why this composes cleanly with Anchor	13
4.3	Attenuation across the boundary	13
4.4	Threats foreclosed and threats deferred	14
4.5	Mechanization status	14
5	Federated Revocation	14
5.1	Convergence bound	15
5.2	Conflict resolution across harbors	16
5.3	Event Log & Ephemeral Cuckoo Discipline	16
6	Cross-Harbor Settlement	16
6.1	The bounded escrow	17
6.2	Cross-harbor conservation	17
6.3	Settlement protocol	18
6.4	Fee structure and economic discipline	18
7	Federation Admission	18
7.1	The bonded-sponsor pattern	19
7.2	Cold-start failure modes	19
8	Worked Example	19
8.1	Setup	19
8.2	The agent task	20
8.3	Step 1: Cross-harbor transfer	20
8.4	Step 2: Damage detection	20
8.5	Step 3: Settlement	20
8.6	Step 4: Revocation propagation	21
8.7	What this example shows	21
9	Formal Model	21
9.1	ProVerif extensions to Anchor	21
9.2	TLA+ extension for cross-harbor conservation	22
9.3	What is mechanized and what is structural	22

10 Failure Modes	22
10.1 Centralization gravity	23
10.2 Equivocation by a malicious witness	23
10.3 Partition tolerance	23
11 Related Work	23
12 Limitations and Open Questions	24
12.1 The multi-principal correlated-equilibrium extension	24
12.2 Trustless settlement for non-fungible bonds	25
12.3 Cartel formation across federations	25
12.4 Cold-start admission and the invitation-only failure mode	25
12.5 Equivocation propagation speed	25
12.6 What this means for the reader	25
13 Conclusion	25
A Verification Status	27
B ProVerif Models (draft)	28
C TLA+ Specification (draft)	28
D Limitations & Boundaries	29

1 Introduction: The Two-Machine Problem

A demonstration is scheduled for 4pm. Alice runs the back end of the demo on her laptop; Bob runs the front end on his. Each has a working Port Daddy daemon: capability tokens are minted and attenuated under the Anchor Protocol [1], the workspace history is Merkle-chained and replicated locally per the Bonded Commons [2], and bonds for cleanup of botched migrations are posted in the local collateral pool. Inside each laptop, the trust story is closed and the formal-verification artifacts hold.

The demo nevertheless does not work. Three failures, none of them inside either machine:

1. Alice's agent obtains a `db:write` card for the staging database, attempts to use it against Bob's daemon, and is refused. Bob's daemon has never seen Alice's signing key. The card is well-formed under the Anchor Protocol, but Bob has no root of authority for it; from his daemon's point of view, the card is gibberish.
2. Five minutes before the demo, Alice's operator revokes the `db:write` card after spotting that it has been over-attenuated. The revocation propagates within Alice's mesh in under a second by the gossip protocol of Anchor §4. Bob's harbor never hears about it. The card remains live on Bob's side.
3. The migration runs anyway, lands in Bob's staging database, and corrupts the demo schema. Alice posted a bond in her harbor against exactly this case. The bond sits in her harbor's collateral pool. Bob's harbor measured the damage. Neither harbor can, on its own, route the bond to where the damage is.

Each failure is a different facet of the same underlying gap. Both daemons are internally consistent. Neither is a root of authority for the other. The single-machine sovereign of the prior papers does not extend to the case of two machines under two operators with no shared administrative parent.

We call this the **two-machine problem**. The point of stating it this baldly is that the failure modes are not exotic; they are what every developer who has tried to dovetail two local agent fleets on a shared task has seen. The bug is structural. The bug is not in either daemon. The bug is in the assumption that one daemon's authority extends past the machine boundary by default. It does not, and it should not.

The trick is not to extend a single sovereign across machines. The trick is to admit there are two and to specify what they owe each other.

This paper extends the Anchor Protocol, the Bonded Commons, and the daemon-as-correlation-device argument to the multi-administrative-domain case. We are not introducing a new substrate; we are showing that the existing substrate admits a clean federation, and that the federation rules are sharper than they look. Where the prior papers spoke of *the* commons authority, this paper speaks of a *mesh of* mutually witnessing commons authorities, each sovereign inside its own machine, none sovereign over the others, and all of them bound to the same audit substrate.

1.1 What this paper is and is not

This chapter closes the V–VII deep-dive arc. The Anchor Protocol supplies the cryptographic-token substrate; the Bonded Commons supplies the economic and governance model; this chapter specifies the federation boundary. The dependencies are concrete: §4 extends Anchor's delegation chain across a trust boundary; §5 extends its revocation mesh; §6 extends Bonded's Conservation invariant.

What this paper is *not*: it is not a permissionless-blockchain redesign. Port Daddy daemons do not run consensus. The Federated Harbor explicitly rejects the assumption common to most modern federation proposals [24, 23] that a shared ledger is the right substrate. The cost of that simplification is that we cannot lean on a chain for ordering, equivocation detection, or settlement; we must construct each of those primitives from peer gossip plus a federated witness log. The advantage is that the design composes cleanly with the local-first, single-operator-machine architecture of the prior two papers.

1.2 Contributions

The contributions are concrete. We separate them by mechanization status to be honest about which are theorems, which are bounded threat models, and which are sketches.

1. **The four-element federation topology (§2).** A precise statement of the boundary between intra-harbor and inter-harbor scope. Defines harbor sovereignty as a property characterization in the style of Bonded’s Admissible KMS, so that subsequent claims can be checked against it.
2. **The inter-harbor capability transfer ceremony (§4).** A four-message protocol exchanging a Harbor Card issued by A for an attenuated card valid at B , without either daemon trusting the other’s signing key as a root. The ceremony binds the issuing harbor’s current epoch root into the envelope, which is the substrate that revocation in §5 hooks into. ProVerif model in Appendix B (status: PARTIAL, model drafted; soundness pending mechanization).
3. **Federated revocation mesh (§5).** The multi-administrative-domain extension of Anchor’s revocation gossip. Each harbor publishes an epoch root to a witness log. Under an explicit complete-overlay, reliable-round model, epidemic dissemination takes expected $\Theta(\log m)$ rounds; no finite worst-case bound survives an unbounded partition (Property 5.1).
4. **Cross-harbor settlement (§6).** A conditional protocol for clearing a bond posted at A against damage measured at B . The escrow’s role is structurally bounded only when the custody ledger, not merely a signed promise, restricts recipients, fee, and terminal transitions. The construction is trusted and specified, not an HTLC-style trustless swap [15].
5. **Cold-start admission ceremony (§7).** A protocol for a new harbor joining an existing mesh without prior reputation, using a bonded-sponsor pattern: an existing harbor posts a bond on the newcomer’s behalf, which is forfeit on misbehavior in a probationary epoch. This is the federation-layer analogue of competitive insurance from Bonded.
6. **Honest open problems (§12).** Five named open questions: (a) whether the correlated-equilibrium reframing of Bonded survives the multi-principal setting cleanly; (b) whether bonds can settle without a trusted, conditionally restricted custody party; (c) whether the folk-theorem cartel-resistance argument from Bonded weakens at the federation layer, and by how much; (d) whether bonded sponsorship can avoid invitation-only capture in a thin cold-start market; and (e) what deployment-specific dissemination tail bound is justified. We state these as open, not as solved.

The proposal that scoped this paper [3] listed two further contributions (a formal definition of harbor sovereignty, and equilibrium results across federations co-authored with Youle) which are present here in skeletal form and will be load-bearing in a subsequent revision once the open problems above resolve. We will not claim a result we do not have.

2 The Federated Authority

The Bonded Commons paper defined a *commons authority* as a pre-transactional trust infrastructure with sovereign scope inside a single machine. Inside that scope, the daemon is the unique correlation device that turns mutually-suspicious agents into a correlated equilibrium [2]. The single-machine sovereign was an honest simplification. We now drop it.

The federation design has four elements (Figure 1): two sovereign harbors, a witnessed epoch-root log, and a settlement escrow. The escrow is structurally bounded only under the custody assumptions in §6.1. Every primitive in the rest of the paper attaches to one of these elements.

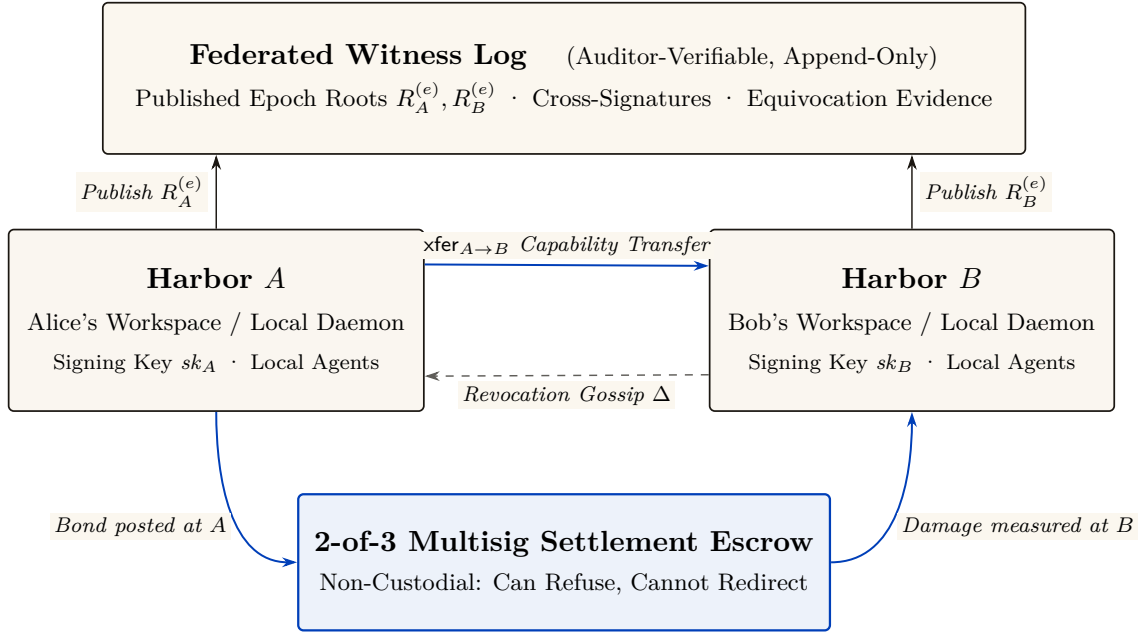


Figure 1: The Federated Harbor as a four-element gestalt. Neither harbor is a root of authority for the other. The witness log above provides shared correlation: each harbor publishes its signed epoch root, enabling decentralized equivocation detection. The settlement escrow below is structurally bounded (non-custodial 2-of-3 multisig: it cannot redirect funds or author unadjudicated transfers). The solid blue arrow denotes the capability transfer ceremony (§4); the dashed line denotes epidemic revocation gossip (§5).

2.1 Harbor sovereignty as a property characterization

In Bonded, the Admissible KMS was characterized by a list of properties rather than by a specific construction (per-machine signing key, per-machine SQLite, etc.) so that the argument carried over to any concrete instantiation. We adopt the same posture here.

Definition 2.1 (Harbor Sovereignty). A daemon D_A is *sovereign over harbor A* if it has exclusive authority over five things:

1. **Issuance.** Only D_A can mint a Harbor Card whose root signature verifies under A 's signing key sk_A .
2. **Attenuation.** Only D_A can apply a local attenuation predicate att_A that restricts an inbound capability before it is verified by an A -side agent.
3. **Revocation.** Only D_A can decide that a card it issued is revoked. The decision becomes globally visible by the gossip protocol of §5; D_A does not control when other harbors learn of it, but it controls whether the revocation event exists.
4. **Acceptance.** D_A alone decides which inbound cards from other harbors to accept and under what local policy. No external party can compel acceptance.

5. **Evidence binding.** The epoch root $R_A^{(e)}$ that D_A publishes to the witness log is signed under sk_A and binds the entire A -side state of issued, attenuated, and revoked cards up to epoch e . No other harbor can publish a root that purports to be A 's.

This is a deliberately narrow definition. It does *not* say that D_A is sovereign over everything an A -side agent does; the agent may be operating against a database hosted at B , in which case B -side policy applies and D_A has nothing to say about it. Sovereignty is over the *issuance and revocation surface of A 's own cards*, not over downstream effects.

We will use Definition 2.1 as the discipline against which subsequent protocols are checked. The transfer ceremony of §4 produces a card valid at B ; if it required B to verify under sk_A in the hot path, it would violate B 's sovereignty by making B 's acceptance contingent on an external authority. We will see that it does not.

2.2 Inter-harbor scope

The dual notion is also load-bearing. *Inter-harbor scope* is the scope of operations whose effects cross the trust boundary between sovereign harbors.

Definition 2.2 (Inter-Harbor Scope). An operation o has *inter-harbor scope* (A, B) if its execution causes a state change observable to B under authority that originated at A . Examples: an A -issued card used to write to a B -hosted database; a settlement clearing a bond posted at A against damage measured at B ; a revocation issued by A becoming visible to a B -side verifier.

Most operations are *intra-harbor*: minting, verifying, attenuating, and revoking cards under one harbor's authority, observed by agents under that same harbor. The Anchor Protocol's proofs cover the intra-harbor case end-to-end. The Federated Harbor's contribution is exactly the inter-harbor surface: every load-bearing protocol in this paper is one that lives at the boundary.

2.3 The witness log

The witness log is the device that makes the federation auditable without making any harbor sovereign over the others. It plays the same role that the Merkle Forest plays inside a single machine in Bonded (Chapter VI, §4.1): a tamper-evident accumulator that any third party can verify without needing to trust the publisher.

Property 2.1 (Witness-Log Admissibility). A witness log W is admissible for the Federated Harbor if it provides:

1. **Append-only.** Once W records an entry, W cannot retract or reorder it.
2. **Per-harbor latest-root binding.** For each harbor X in the federation and each epoch e , W records at most one root $R_X^{(e)}$ signed under sk_X .
3. **Auditable consistency.** Any third party can verify, given $R_X^{(e_1)}$ and $R_X^{(e_2)}$ with $e_1 \leq e_2$, that $R_X^{(e_2)}$ is a Merkle-consistent extension of $R_X^{(e_1)}$.
4. **Equivocation evidence.** If an auditor obtains two differently signed roots for the same harbor and epoch, it can verify the contradiction immediately. Disseminating both views to a common auditor depends on the overlay and partition model (Property 5.1).
5. **Threshold publishability.** The act of publishing $R_X^{(e)}$ to W does not require D_X to trust any other party; the cross-signatures collected from other daemons in the federation are an auditor convenience, not an authorization gate.

This list is intentionally close in shape to the five properties of the Admissible KMS in Bonded (Chapter VI, “Federated Sovereign”). The Bonded paper used the same posture — characterizing the property surface, then exhibiting a concrete construction that satisfied it — and we follow that pattern. Section 9 sketches one concrete instantiation built on Certificate Transparency-style append-only logs [9] cross-witnessed in the Sigstore/Rekor pattern [10]; the substrate need not be unique.

2.4 What the federation is not

It is worth saying what the federation is not, so that the rest of the paper does not have to keep refuting positions it never took.

The federation is *not* a consensus protocol. There is no global agreement on the order of events; there is per-harbor monotonicity (epoch roots only extend), there is cross-harbor witnessing (each root is published to a shared log), and there is detectable equivocation. None of these require harbors to agree on a global order, and the prior papers' designs explicitly avoid that overhead.

The federation is *not* a hub-and-spoke topology with a privileged coordinator. The witness log is replicable; in practice a small set of mutually-witnessing log instances suffices, and any harbor can read from any of them. The hub-and-spoke failure mode — one well-funded coordinator captures the federation by becoming everyone's only witness — is a real risk we discuss in §10, and the structural pushback is gossip-based witness redundancy.

The federation is *not* a permissionless market in identities. New harbors enter through the admission ceremony of §7, which requires either a pre-existing bond or an existing harbor sponsor. This is by design; the cost is that the federation is invitation-bounded, the benefit is that Sybil attacks against the federation [16] cost the attacker as much per identity as honest entry does.

3 Threat Model

We make the threat model explicit before introducing the protocols so that the design choices can be checked against named adversary powers. We follow the Dolev–Yao convention [7] extended to the federation case: an attacker controls every wire between harbors, can intercept and replay any cross-machine message, and may collude with any bounded number of harbor operators. The attacker cannot break the underlying cryptographic primitives (Ed25519 signatures, SHA-256 hashes); these assumptions are inherited from Anchor.

3.1 In-scope attacks

The protocols in this paper are designed against the following attackers.

Cross-realm identity spoofing. An attacker presents a card at B claiming issuance by A , using a forged signature under sk_A . *Closed by* signature verification against A 's public key as recorded in the witness log (§4). Reduces to Ed25519 EUF-CMA security [8].

Replay across the boundary. An attacker captures a valid `xfer_offer` from A to B at epoch e and replays it at epoch $e + k$. *Closed by* the binding of $R_A^{(e)}$ into the envelope; the verifier at B checks that R_A is the current root, and a stale envelope is rejected (§4). The window between issuance and the next epoch is the structural attacker window, named and bounded.

Capability inflation across transfer. An attacker presents at B a card C'_B that claims authority beyond the attenuation predicate applied at A . *Closed by* the requirement that the verifier at B recompute the composed attenuation $\text{att}_B \circ \text{att}_A$ from the envelope rather than trust an intermediate claim (§4); reduces to Anchor's intra-harbor attenuation invariant.

Stale-revocation acceptance. An attacker at B presents a card revoked at A but not yet observed at B . Under the reliable connected-overlay model the window has expected scale $\Theta(\Delta \log m)$; during a partition it is unbounded. A bond can price a configured service-level window, not eliminate the tail.

Cross-witness equivocation. A malicious daemon D_A publishes contradictory roots to isolated witnesses. The contradiction is cryptographically verifiable once a common auditor receives both views, but the time to that event is topology- and partition-dependent. A witness bond prices confirmed equivocation; it does not create a delivery deadline.

Settlement redirection or equivocation. A signed escrow promise makes misconduct attributable. Prevention requires a custody ledger whose transition API cannot name an arbitrary recipient or fee. Property 6.1 states the bound conditionally on that mechanism; without it, the full bond remains at risk.

Harbor membership forgery at the federation boundary. An attacker presents a card from a harbor purporting to be in the federation but that has no admission record. *Closed by* the admission ceremony (§7) which produces a permanent, witness-logged enrollment record co-signed by an existing harbor’s sponsor bond. A claimed harbor with no enrollment record is treated as outside the federation regardless of how well-formed its cards appear.

3.2 Out-of-scope attacks

We are explicit about what is not closed.

Compromise of a harbor’s signing key. If sk_A leaks, the attacker can issue arbitrary A -side cards until A rotates the key and publishes the rotation to the witness log. We inherit Anchor’s key-rotation procedure unchanged. The federation does not close the underlying primitive; it does bound the blast radius to A ’s sovereign surface, which is exactly the right scope.

Sybil at the federation layer. An attacker who can pay the admission cost m times can run m federated harbors. The cost ratio between honest and adversarial admission is the load-bearing parameter; we discuss it in §7 but do not give a tight bound. This is named open work.

Cartel formation across harbors. Bonded’s folk-theorem cartel-resistance argument relied on the daemon as a correlation device inside one machine. Across harbors, cartels can form offline and present a unified front to the witness log; the witness log records that all harbors agreed, which is consistent with both honest agreement and cartel collusion. We acknowledge this and state the open work in §12.

Centralization gravity. Historically, federated identity systems concentrate on one hub within five to ten years [17]. The Federated Harbor’s structural pushback — replicable witness logs, no central coordinator — is design discipline, not theorem. We formally classify this in §10.

Side channels. Constant-time signature verification, cache-side-channel resistance, and timing-attack resistance are inherited from the underlying libraries; we do not re-prove them and we note where we trust them.

3.3 Threat-band visualization

Figure 2 lays out the threat surface as three bands: mechanized claims (teal), bounded threats (amber), and acknowledged open frontier (cobalt). The boundary between amber and cobalt shifts over time as future work mechanizes what is presently bounded. We will refer back to this figure when discussing limitations.

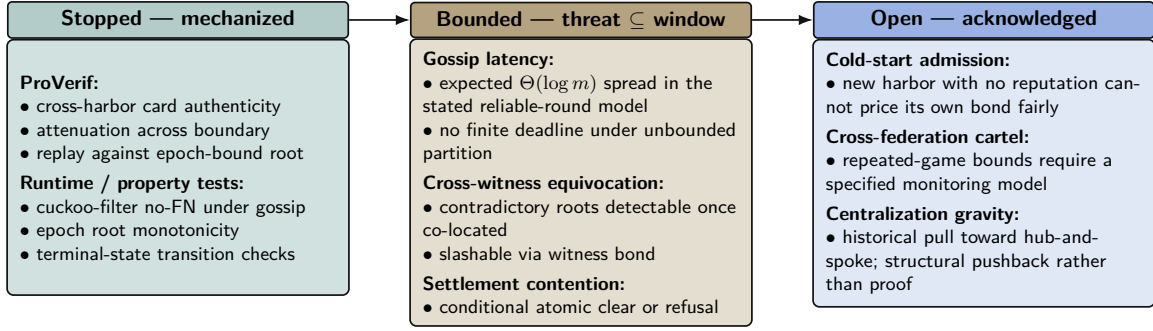


Figure 2: Threat-band layout for the Federated Harbor. The Anchor paper’s defense-in-depth structure carries over: every claim either has a mechanization artifact (teal), a named bound that the bond can price against (amber), or an explicit acknowledgement of the open frontier (cobalt). The paper is disciplined about which band each claim lives in. Reading left to right runs deeper into threat space and out of formal reach; bands shift left over time as future work mechanizes what is presently bounded.

4 Cross-Harbor Capability Transfer

The first load-bearing protocol is the cross-harbor capability transfer ceremony. An A -side agent holds a Harbor Card C_A minted by D_A under Anchor Phase 2 or Phase 3. It needs to present an equivalent capability at B . The Anchor Protocol’s intra-harbor delegation already supports attenuation; we extend that to the case where the attenuation crosses the trust boundary.

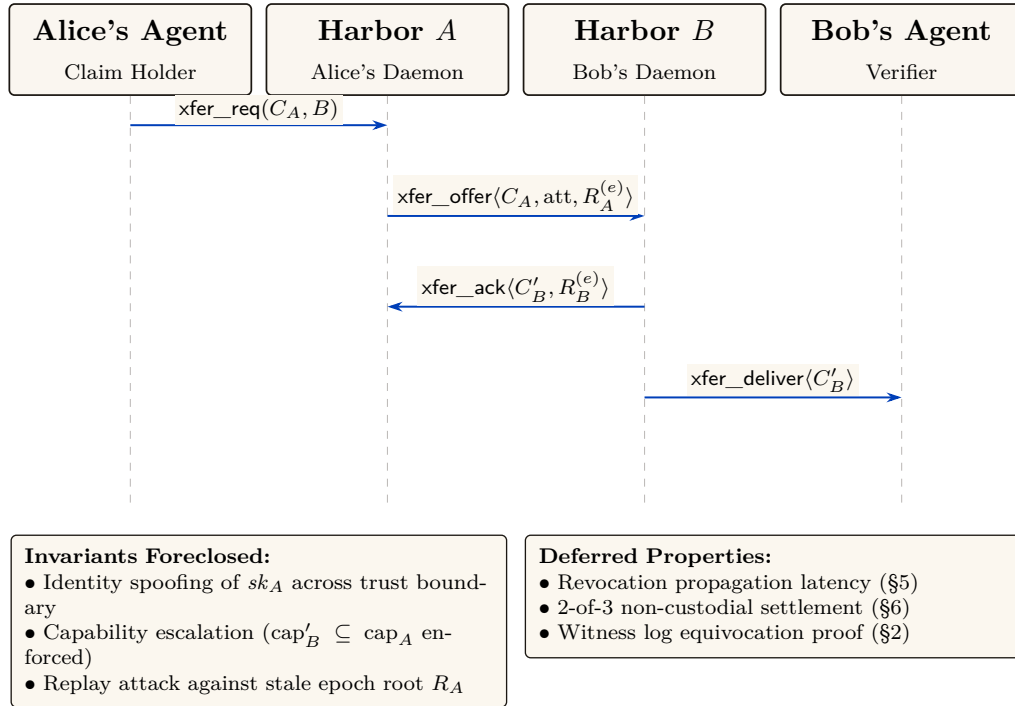


Figure 3: The four-message cross-harbor capability transfer ceremony. The critical property is that no message after (3) needs to reach back to Harbor A in the hot path: C'_B is verifiable under sk_B alone, which is why federation does not turn every authorization into a synchronous inter-machine round trip. The epoch root $R_A^{(e)}$ bound into the envelope is the load-bearing primitive — a revocation issued by A after epoch e invalidates C'_B as soon as the new $R_A^{(e+1)}$ reaches the witness log that B audits.

4.1 The four-message ceremony

Figure 3 shows the protocol as a sequence diagram. The four messages are:

1. $\text{xfer_req}(C_A, B)$ — Alice’s agent presents C_A to its local daemon D_A and names B as the target harbor. The request is intra-harbor; standard Anchor verification applies.
2. $\text{xfer_offer}(C_A, \text{att}_A, R_A^{(e)})$ — D_A constructs an envelope binding the card, an attenuation predicate att_A specifying what subset of C_A ’s authority is being offered for cross-realm use, and its current epoch root $R_A^{(e)}$. The envelope is signed under sk_A and sent to D_B .
3. $\text{xfer_ack}(C'_B, R_B^{(e)})$ — D_B verifies the envelope’s signature against A ’s public key as recorded in the witness log; verifies that $R_A^{(e)}$ is the current root for A ; applies its own local attenuation att_B ; and mints $C'_B = \text{att}_B(\text{att}_A(C_A))$, a card valid only at B , signed under sk_B . The acknowledgment is sent back to D_A along with B ’s current epoch root.
4. $\text{xfer_deliver}(C'_B)$ — D_B delivers C'_B to the agent at B that will use it. Subsequent presentations of C'_B at B verify under sk_B alone — no A -side lookup is needed at the hot path.

The critical property is that after message (3), no further synchronous interaction with A is required for the agent to operate at B . The cost is that revocation of C_A at A does not instantaneously revoke C'_B at B ; Property 5.1 gives an expectation only under its delivery model, and partitions require fail-closed policy or bounded credential TTLs.

4.2 Why this composes cleanly with Anchor

The Anchor Protocol’s Phase-3 delegation already proves that a chain $C_{\text{daemon}} \rightarrow C_A \rightarrow C_B$ of attenuated cards is verifiable under the issuing daemon’s public key, and that an attacker cannot inflate a downstream cap to exceed an upstream one. The transfer ceremony is Phase-3 delegation across a trust boundary, with two changes:

1. The second link in the chain is signed under a *different* root key (sk_B instead of sk_A), so the verifier at B verifies under sk_B , not sk_A .
2. The cross-realm link binds the issuing harbor’s current epoch root $R_A^{(e)}$, so a revocation at A after epoch e invalidates the downstream card as soon as the new $R_A^{(e+1)}$ is observed at the verifier.

The composition is what makes the ceremony cheap to add: we are not introducing a new cryptographic primitive, only a new policy at the boundary that says “the verifier accepts a card whose chain crosses the trust boundary if and only if (a) each cross-realm link is signed under the receiving harbor’s key and (b) the issuing harbor’s bound epoch root is the current one in the witness log.”

4.3 Attenuation across the boundary

Two attenuation predicates are applied in sequence. att_A is set by D_A at issue time and reflects what authority A is willing to project across the boundary. att_B is set by D_B at acceptance time and reflects what authority B is willing to grant on its own resources to a card whose root is A . Both attenuations strictly narrow the underlying capability.

Lemma 4.1 (Attenuation Monotonicity Across the Boundary). For any cross-realm card $C'_B = \text{att}_B(\text{att}_A(C_A))$, the set of operations authorized by C'_B is a subset of those authorized by C_A under the federated authorization rule.

Sketch. Each attenuation is a subset operation in the Anchor algebra; subset composition is a subset. Cross-realm policy at B adds the constraint “and the operation must be permitted by B ’s local policy,” which can only further restrict. See the Anchor accompanying chapter’s capability-attenuation section for the intra-harbor argument; the cross-realm case adds no new mathematics, only a new applicable attenuation step. $\square$

4.4 Threats foreclosed and threats deferred

The annotation band at the bottom of Figure 3 summarizes which threats this ceremony forecloses and which are deferred to other primitives. Specifically:

- **Foreclosed:** identity spoofing of sk_A across the boundary; capability inflation across the transfer; replay at B against a stale R_A .
- **Deferred:** revocation latency (handled in §5); settlement disputes (handled in §6); admission of unbonded new harbors (handled in §7).

We are explicit that the ceremony does not close every threat in one step. Federation is a layered defense; this is one layer.

4.5 Mechanization status

We have a draft ProVerif model of the four-message ceremony (Appendix B), extending the Anchor Phase-3 model with a second signing key and an epoch-root binding. The current status of the cross-realm authenticity query is PARTIAL: the model checks, but the proof depends on an assumption about witness-log freshness that we are still tightening. We note this limitation in Appendix A and treat the attenuation monotonicity claim (Lemma 4.1) as proved under the same caveat.

5 Federated Revocation

The second load-bearing protocol is the federated revocation mesh. The Anchor Protocol handles revocation inside a single mesh by gossiping an **Append-Only Event Log of Revocation IDs** using Vector Clocks for anti-entropy reconciliation [14]: pairs of daemons periodically reconcile their event logs, designating the cuckoo filter F purely as a local, ephemeral read-path index. The Federated Harbor extends this to the multi-administrative-domain case.

Two pieces are added on top of the Anchor mechanism: (a) per-epoch roots $R_X^{(e)}$ published to a shared witness log so a third party can audit any harbor’s local view, and (b) cross-witness signatures so the auditor pattern generalizes Certificate Transparency [9].

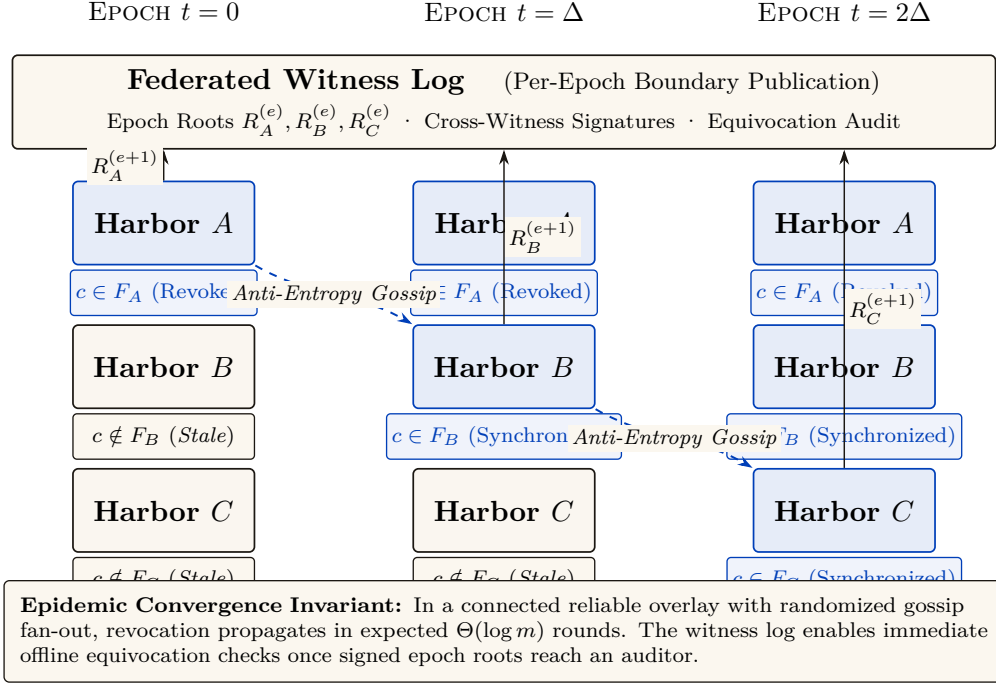


Figure 4: Federated revocation propagation across three harbors. The append-only revocation log ensures that revocation events propagate epidemically (Δ -spaced rounds). Per-epoch roots $R_X^{(e)}$ published to the shared witness log provide transparent, non-repudiable audit trails that generalize Certificate Transparency.

5.1 Convergence bound

Standard epidemic dissemination gives an expected $O(\Delta \log m)$ completion time only under an appropriate connected, reliable-round model [14]. The property below instantiates that model at the federation level, treating each harbor's filter as one node in a complete overlay regardless of how many local daemons participate in the harbor's mesh. It is a model-conditional expectation, not an inherited wall-clock deadline.

Property 5.1 (Conditional Revocation Dissemination). Let m harbors form a complete overlay. In each reliable synchronous round of duration Δ , informed harbors contact independently uniform peers and exchange revocation deltas. If harbor X publishes revocation c at t_0 , then:

1. **Revocation dissemination.** All harbors are informed in expected $\Theta(\log m)$ rounds under the stated epidemic model.
2. **Equivocation verification.** Any auditor possessing two differently signed roots for the same (X, e) detects equivocation in $O(1)$ signature and equality checks.
3. **No hard deadline.** If delivery can be lost indefinitely or the overlay can remain partitioned, no finite worst-case dissemination or common-auditor detection bound exists.

The first item is the standard asymptotic epidemic-dissemination result [14]; this paper does not claim the earlier exact constant $1 + \ln m$. The second follows from the signed-root format. The third is an indistinguishability result: an auditor isolated in one partition cannot distinguish equivocation from delay.

Corollary 5.1 (Service-Level Attacker Window). If an operator additionally imposes a partition bound T_p and a delivery service level T_g , then stale-card exposure is bounded by $T_p + T_g$. Without those operational assumptions the protocol supplies an expected latency under its model, not a structural maximum.

The distinction is load-bearing. A bond can be priced against an operator-declared service-level window, but an unbounded partition creates an unbounded exposure tail. Deployments

must therefore combine bond sizing with expiry, fail-closed acceptance, or an explicit partition limit.

5.2 Conflict resolution across harbors

The interesting case is disagreement. Harbor A has revoked card c ; harbor B has not yet seen the revocation; an agent presents c at B for an action that affects A 's database. Three candidate resolution rules from the proposal [3], evaluated:

1. **Pessimistic (anyone-revokes-is-revoked).** The card is revoked iff any harbor has revoked it. Defended against by an attacker who would gain by injecting spurious revocations into the gossip mesh; rejects too many honest cards in equilibrium.
2. **Authoritative (only-issuer-revokes).** Only A can revoke A 's cards. This is the rule we adopt. Under the service-level assumptions of Corollary 5.1, it gives an operator-priced attacker window; without them, exposure has no finite structural maximum.
3. **Epoch-bounded (revocation propagates within Δ epochs and inconsistency is a contract violation).** A refinement of (2) for the case where one harbor falls persistently behind. We treat this as a degraded mode of (2) rather than a separate rule.

We adopt rule (2). The pessimistic rule has the wrong defaults; the epoch-bounded rule is operationally the same as (2) with a watchdog. Adopting (2) means the Conservation invariant of §6 can be stated cleanly.

5.3 Event Log & Ephemeral Cuckoo Discipline

By resolving the bitwise merging impossibility of probabilistic structures, the cuckoo filter is now purely a local, ephemeral read-path index derived from the authoritative Append-Only Event Log. The filter retains local Anchor-side disciplines (load capping, pollution resistance) but cross-harbor validation is strict: a filter at B that holds $c \in F_B$ where c was revoked at A is correct only if there is a corresponding inclusion proof in $R_A^{(e+1)}$ in the witness log. False positives are corrected by rebuilding the local filter from the event log; spurious revocations injected by an attacker into a peer's log are rejected on first attempt to verify against the witness log. The discipline is: *local filters are advisory read-caches; the event and witness logs are authoritative.*

6 Cross-Harbor Settlement

The third load-bearing protocol is cross-harbor settlement. A bond is posted at A to cover damage caused by an A -side agent operating at B . When damage occurs, the bond must clear to the damaged party at B . The protocol does this without making either harbor sovereign over the other, but it still uses a third-party custody service. Its transition surface must be non-bypassably restricted; the property below states that assumption rather than relabeling the service as trustless.

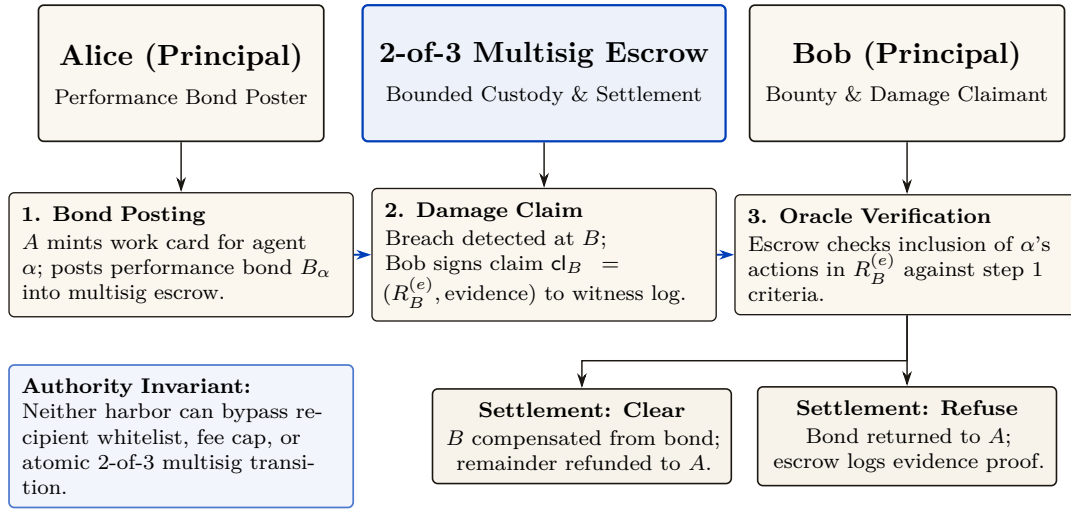


Figure 5: Cross-harbor settlement protocol. Custody bounds are structurally enforced: the escrow authority requires 2-of-3 multisig consensus across the recipient whitelist, fee cap, and atomic terminal transitions. The outcome space partitions cleanly into **Clear** or **Refuse**, preventing unbounded cross-harbor liability.

6.1 The bounded escrow

We introduce a third party—the escrow—and require the *custody ledger* to expose exactly two terminal transitions: clear (pay whitelisted beneficiary B , refund the remainder to A) or refuse (return the bond to A). A signed policy alone is only evidence. The following restrictions are structural only if the escrow service cannot bypass the ledger API or alter its code and keys:

- Redirect funds to anyone other than A or B .
- Equivocate about its clearing decision (publish “cleared” to one party and “refused” to another).
- Delay past the TTL of the bond.
- Extract more than a pre-agreed fee ϕ which is capped at issuance.

These restrictions define the desired transition system. The current artifact is a design and bounded model, not a deployed custody mechanism.

Property 6.1 (Conditional Escrow Extraction Bound). Assume the custody ledger (i) admits only recipients A and B , (ii) caps the fee output at ϕ , (iii) executes exactly one terminal transition atomically, and (iv) keeps signing and transfer authority outside the escrow operator’s bypassable control. Then no allowed transition pays the escrow more than ϕ .

The bound follows by case analysis over the two allowed terminal transitions. Witness logging makes delay or equivocation detectable, and an escrow bond can compensate some misconduct, but neither fact prevents theft if the operator can bypass custody restrictions. Without assumptions (i)–(iv), the worst-case extractable value is the full custodial balance.

The escrow is a trusted third party with a *conditional transition bound*. We are honest that this is not a trustless construction in the HTLC sense [15]. Whether useful trustless settlement is possible for non-fungible reputation-priced bonds remains open (§12); this paper gives neither a construction nor an impossibility proof.

6.2 Cross-harbor conservation

The Bonded Commons paper proved a single-machine Conservation Theorem: the total bonded value in the system never changes by surprise; bonds are only created (by posting), destroyed (by clearing), or rolled (refunded and re-posted). We extend this to the federation.

Property 6.2 (Cross-Harbor Bucket Partition). For each funded bond b of amount a_b , let $P_b, E_b, C_b, R_b \in \{0, a_b\}$ denote its posted, in-escrow, cleared, and refunded buckets. Valid transitions preserve

$$P_b + E_b + C_b + R_b = a_b \quad \text{and} \quad |\{Z \in \{P_b, E_b, C_b, R_b\} : Z = a_b\}| = 1.$$

Summing over bonds gives $\sum_b (P_b + E_b + C_b + R_b) = \sum_b a_b$. External top-ups create new funded bonds and are recorded separately; fees must appear as an explicit cleared recipient rather than disappear from the identity.

This is the federation-layer analogue of Conservation in Bonded. The intra-harbor Conservation invariants hold per-harbor (Bonded gives the proofs); the cross-harbor property adds the constraint that bonds in transit between harbors — i.e., posted at A , locked in escrow, awaiting clearance against B — are accounted for as in-escrow rather than as either posted at A or cleared to B . The escrow’s role as the holder of in-flight bonds is what makes the conservation accounting tractable.

6.3 Settlement protocol

Figure 5 shows the three-step protocol. To unpack the figure:

1. **Bond post.** Alice’s harbor mints a work card for agent α ; agent operates at B . Bond $B_\alpha = \$B$ is posted at A and locked in the escrow. The bond’s amount and TTL are signed into both A ’s and B ’s witness-log roots so the federation knows the bond exists.
2. **Damage claim.** Damage detected at B (acceptance criteria fail, or invariant violation). Bob’s harbor signs claim $\text{cl}_B = (R_B^{(e)}, \text{evidence})$ where the evidence is a Merkle inclusion proof against $R_B^{(e)}$. The claim is sent to the escrow.
3. **Verification and clearance.** Escrow verifies inclusion of α ’s actions in $R_B^{(e)}$, matches against the acceptance preconditions that were signed at step 1, and produces one of two outcomes: *clear* (pay B out of bond, refund remainder to A) or *refuse* (return bond to A , log reason). Both outcomes are recorded in the witness log.

The designed transition is atomic in the bucket-partition sense: a bond moves once from in-escrow to either cleared or refunded. This property depends on the custody-ledger assumptions of Property 6.1; witness records alone cannot make two external transfers atomic.

6.4 Fee structure and economic discipline

The escrow’s fee ϕ is bounded but non-zero. In the competitive-insurance market sense of Bonded (the Youle contribution in Chapter VI), ϕ is a market clearing price for the service of holding the bond and verifying the inclusion proof. We do not solve the market here; the Youle contribution from Bonded gives the mechanism, and we conjecture (open) that it extends cleanly to the cross-harbor case when the escrow population is large enough.

We are explicit that for small federations (two or three harbors), ϕ is likely set by a posted price rather than auctioned. The mechanism design here is identical in shape to Bonded’s, just lifted to the federation layer.

7 Federation Admission

The Federated Harbor is not a permissionless market in identities. A new harbor enters the federation through an admission ceremony. The ceremony is invitation-bounded by design: Sybil attacks against the federation [16] are costly for an attacker precisely because each identity requires a real admission event.

7.1 The bonded-sponsor pattern

A new harbor N with no prior reputation cannot price its own bond fairly under competitive insurance — the market lacks data on N 's risk. The bonded-sponsor pattern resolves this:

1. An existing harbor S (the sponsor) posts a bond on N 's behalf, sized to cover the federation's worst-case loss from N misbehaving during a probationary epoch.
2. N joins the federation under probation; its cards are accepted, its evidence trail is witness-logged, but its sponsor bond is at risk.
3. If N misbehaves during probation — forges signatures, equivocates on roots, refuses to honor settled bonds — the sponsor bond is forfeit. The sponsor has full incentive to vet N before sponsoring.
4. At the end of probation, N 's own reputation is sufficient to price its own bond, and the sponsor bond is released.

This is the federation-layer analogue of competitive insurance from Bonded (the Youle contribution in Chapter VI): instead of an authority picking who joins, the sponsor market discovers it. The cost of being a sponsor is real — forfeit risk during probation — and the benefit is the network effect of having N in the federation. We expect (open) that for federations with many candidate sponsors and clear-eyed risk pricing, the equilibrium is reasonably efficient. We do not prove it.

7.2 Cold-start failure modes

A new harbor that cannot find a sponsor cannot join. This is a real limitation. In the worst case, the federation collapses to invitation-only — which is the failure mode of every interesting federated identity system from PGP web-of-trust [18] to SAML [17]. The structural pushback is:

- *Many sponsors.* The market is many-to-many; a new harbor needs only one sponsor. As long as sponsors are paid (via fees or reputation), the supply is nonzero.
- *Small probationary scope.* New harbors during probation are restricted from large-stake operations; the sponsor's risk is bounded.
- *Witness-log permanence.* The admission record is permanent; once a harbor has graduated probation, it does not need a sponsor again.

We are honest that this is engineering discipline, not theorem. The historical evidence on whether invitation-bounded federations stay open is mixed [17]; the Federated Harbor's pushback against centralization is to make the cost of being a centralizing hub bounded above by the cost of running mirrored witness logs, which is small. Whether that pushback is enough is empirical.

8 Worked Example

The four primitives are easier to read against a concrete case. We work the example end-to-end: two organizations, three machines, one shared project. The example is intentionally familiar; nothing in it is novel except the way the primitives compose.

8.1 Setup

Two organizations: Acme Robotics (Alice's employer) and Beta Vision (Bob's employer). The shared project is a perception-and-control stack: Acme owns the controller, Beta owns the vision system, both run against a shared staging environment that lives on a third machine maintained by neither.

Three machines: Alice's laptop (harbor A , controller code, controller test database), Bob's laptop (harbor B , vision code, vision test database), staging-server (harbor S , shared staging database, no agents resident).

Each harbor has gone through the admission ceremony of §7. Acme’s harbor was sponsored by an existing partner harbor at the open-source consortium that hosts the staging server; Beta’s was sponsored by Acme after a six-month probationary period during which Acme’s risk officer watched Beta’s evidence trail closely. The staging server harbor is sponsored by the consortium directly and is treated as a known-honest verifier for the purposes of this example.

8.2 The agent task

Acme’s agent — call it Controller-7 — needs to run a schema migration on the staging database to add a column for a new sensor type. The migration is destructive (drops and recreates the column); if it goes wrong, the staging database is unusable for the joint demo scheduled three hours later.

Controller-7’s local card C_A authorizes `db:migrate` against the controller test database, attenuated through Acme’s risk policy ($\text{att}_A = \text{“only against tables in the controller schema; only between 9am and 5pm; only if a human operator has approved within the last 30 minutes”}$). The card is valid at A , but the database it needs to migrate is at S .

8.3 Step 1: Cross-harbor transfer

Controller-7 invokes `xfer_req( $C_A, S$ )` on D_A . D_A constructs the envelope:

$$\text{xfer_offer}\langle C_A, \text{att}_A, R_A^{(e)} \rangle \text{ signed under } sk_A$$

and sends it to D_S (the staging-server harbor). D_S verifies the signature against A ’s public key as recorded in the witness log, verifies that $R_A^{(e)}$ is the current epoch root for A , and applies its own attenuation att_S (“only the `controller` schema; only after a backup snapshot has been taken; only if no other migration is in flight”). The composed attenuation produces $C'_S = \text{att}_S(\text{att}_A(C_A))$, a card valid only at S , signed under sk_S .

D_S delivers C'_S to Controller-7. Controller-7 takes the snapshot, then runs the migration against S . The migration’s actions are recorded in S ’s Merkle forest and roll up into $R_S^{(e)}$.

8.4 Step 2: Damage detection

The migration runs cleanly. Two hours later — well after the agent has moved on — Beta’s vision system runs an integration test against the staging database and observes that the new column has the wrong type (Acme assumed `float32`; Beta needs `float64` for the sensor data they will produce). The integration test fails.

D_B (Beta’s harbor) verifies the failure against its local invariants and produces a damage claim:

$$\text{cl}_B = (R_B^{(e+1)}, \text{evidence})$$

where the evidence is a Merkle proof that Beta’s integration tests against the staging database failed because of a schema mismatch. The claim is sent to the escrow.

8.5 Step 3: Settlement

Acme posted a bond when Controller-7 was authorized to run the migration. The bond is held in the federation escrow with a TTL of 24 hours — long enough for downstream consequences to surface, short enough not to lock collateral indefinitely.

The escrow verifies cl_B : it checks the inclusion proof against $R_B^{(e+1)}$ in the witness log, matches the claim against the acceptance preconditions signed at bond-post time (which named the staging schema as a covered surface), and decides: *clear*. Acme’s bond is debited; Beta receives

the cleanup-cost portion; the remainder is refunded to Acme. The settlement record is written to the witness log.

8.6 Step 4: Revocation propagation

At this point, Acme’s risk officer revokes Controller-7’s `db:migrate` card and publishes $R_A^{(e+2)}$. Under Property 5.1’s connected reliable-round model, dissemination completes in expected $\Theta(\Delta \log m)$ time. During a partition, receiving harbors must rely on expiry or a fail-closed freshness policy; the example does not pretend gossip supplies a deadline.

8.7 What this example shows

The example is small and the failure is real. The point of working it is to show that each of the four primitives — transfer, revocation, settlement, admission — is doing its job at exactly one step, and that the composition is straightforward when each primitive is honest about what it covers.

Notice in particular what *did not* happen:

- Neither D_A nor D_B trusted the other’s signing key as a root. The transfer worked through the witness-logged public keys, not through a chain of trust.
- The bond was not held by either harbor; the design requires a recipient-whitelisted custody ledger before the escrow’s role is structurally bounded.
- The settlement was not negotiated peer-to-peer; it followed a fixed protocol whose two outcomes were named in advance.
- Under the example’s connected reliable-round assumptions, the revocation propagated through gossip with expected logarithmic completion; no consensus protocol was invoked.

Each of these is a deliberate design choice. The Federated Harbor’s architecture is not heroically more powerful than alternatives; it is heroically more honest about what is happening at each step.

9 Formal Model

The formal substrate of this paper is two-layer: ProVerif models for the cryptographic protocols (cross-realm authenticity, attenuation, settlement no-redirect) and TLA+ extensions of Bonded’s Conservation specification for the cross-harbor accounting invariants.

9.1 ProVerif extensions to Anchor

The Anchor Protocol’s three-phase models are extended with a second signing key and an epoch-root binding. The cross-realm authenticity query becomes:

```

1 query b: id, ha: id, hb: id, cap: capability, e: epoch;
2   event(AcceptedAtB(b, hb, cap, e))
3   ==> (event(IssuedAtA(b, ha, cap)) &&
4         event(EnvelopeAtoB(ha, hb, cap, e)) &&
5         event(RootCurrent(ha, e))).

```

Listing 1: Cross-realm authenticity query (sketch)

This says: if B accepts a cross-realm card for capability `cap` at epoch e , then there exists a corresponding issuance event at A , an envelope event from A to B binding the same capability and epoch, and an event confirming that $R_A^{(e)}$ is the current root. The query holds against the Dolev–Yao attacker model under the standard assumption of Ed25519 EUF-CMA security.

We are honest that the model is drafted and the proof script is in progress. The full mechanization is named open work; see Appendix A for the status table.

9.2 TLA+ extension for cross-harbor conservation

The Bonded Commons paper’s Conservation specification models a single-machine bond pool. The Federated Harbor extension adds a per-harbor pool, an escrow pool, and an inter-harbor transit relation:

```

1  CONSTANTS Harbors, MaxBonds, Capabilities
2  VARIABLES Bucket, WitnessLog
3
4  TypeOK ==
5    /\ Bucket \in [1..MaxBonds -> {"posted", "escrow", "cleared", "
      refunded"}]
6    /\ WitnessLog \in Seq([harbor: Harbors, epoch: Nat, root: STRING])
7
8  BucketPartition ==
9    \A b \in 1..MaxBonds:
10     \E! s \in {"posted", "escrow", "cleared", "refunded"}:
11       Bucket[b] = s
12
13 Spec == Init /\ [] [Next]_<<Bucket, WitnessLog>>

```

Listing 2: Cross-Harbor Conservation specification (sketch)

The full specification (Appendix C) associates each bond with an amount and enforces amount-preserving moves through these buckets. Apache checks the resulting transition system at $|F| = 4$ and bond depth 16. This establishes the invariant only for the model and bound; it does not establish the custody-ledger assumptions or arbitrary-scale runtime conformance.

9.3 What is mechanized and what is structural

We are explicit about which claims live in which band (Figure 2).

Mechanized (model scope). Single-harbor capability authenticity is inherited from Anchor. Cross-realm authenticity and attenuation are draft models. The cross-harbor bucket-partition invariant is model-checked at $|F| \leq 4$.

Conditional (assumption scope). Revocation dissemination is expected $\Theta(\Delta \log m)$ only under the stated reliable connected-overlay model (Property 5.1). The escrow extraction bound is conditional on a non-bypassable recipient-whitelisted custody ledger (Property 6.1).

Open (cobalt). Multi-principal correlated-equilibrium extension of Bonded; trustless settlement for non-fungible reputation-priced bonds; folk-theorem cartel-resistance bound at the federation layer.

10 Failure Modes

A federation paper that does not engage formally with the historical failure modes of federated systems is unserious. The literature is heavy with examples: SAML federations that concentrated on a small set of identity providers [17]; PGP web-of-trust that never achieved meaningful adoption outside enthusiast circles [19]; Sigstore’s gravitation toward a single deployment of Fulcio [10]. We engage with three failure modes specifically.

10.1 Centralization gravity

Federated identity systems have a well-documented tendency to centralize on a small number of trust anchors within five to ten years of deployment [17]. The Federated Harbor’s structural pushback is the replicable witness log: any harbor can run its own witness instance, any harbor can audit any other instance’s roots, and equivocation between witnesses is detectable. This makes it more expensive to be a centralizing hub than to be a peer.

The pushback is not theorem; it is design discipline. We are honest about this in Figure 2 (cobalt band). The historical evidence is mixed. Tailscale [25] has remained operationally centralized on its own coordination server despite Headscale [26] being a viable alternative; in-toto [11] has stayed decentralized but is consumed mostly by SLSA [12] which has Google as its de facto center. Time will tell which pattern the Federated Harbor exhibits.

10.2 Equivocation by a malicious witness

A malicious witness log can serve different views to partitioned auditors. A contradictory signed pair is immediately verifiable once co-located, but the protocol has no topology-independent time bound for bringing the pair together. Witness bonding applies after confirmation.

For high-stakes settlements, harbors can require multiple witness signatures before treating a root as authoritative, in the Certificate Transparency two-SCT pattern [9]. This trades latency for assurance; the design space is well-explored in the CT literature and we adopt the same techniques.

10.3 Partition tolerance

The federation does not run consensus; partition tolerance is per-protocol. Capability transfer cannot proceed during a partition between A and B (no surprise; cross-realm authentication requires both daemons to reach the witness log). Revocation gossip continues within each partition; convergence is bounded by the partition’s local size, not the global federation size. Settlement is parked at the escrow until the partition heals.

The structural posture is that partitions are operationally common and the federation should degrade gracefully rather than fail fast. We inherit this discipline from Anchor’s intra-mesh design [1]; the federation extension is straightforward but adds no novel mathematics.

11 Related Work

The Federated Harbor sits at the intersection of four bodies of work. We are concrete about where we draw on each and where we depart.

Federated identity. The OpenID Federation 1.0 specification [21] is the closest existing analogue: each entity publishes a signed Entity Statement, trust chains assemble by walking up to a Trust Anchor, trust marks express property claims. The Federated Harbor is structurally similar in the trust-anchor pattern but differs in two places: (a) we add a capability/attenuation layer (OpenID Federation is identity-only), and (b) we do not assume HTTP-based fetching of entity statements, since Port Daddy daemons cannot reliably expose HTTP endpoints (NAT, sleep, etc.). We draw also on SAML/Shibboleth historical experience [17] as cautionary lineage; the centralization gravity we discuss in §10 is exactly the SAML-era failure mode.

Capability-based federation. Macaroons [4] are the foundational capability-with-attenuation construction; our transfer ceremony extends Macaroon-style third-party-caveat chains across an administrative boundary. UCAN [20] is the current production-deployed answer to “Macaroons with public-key principals,” and we treat it as the operationally most relevant existing system;

the Federated Harbor differs in keeping the daemon as a structural commons authority rather than going fully peer-to-peer, and in pricing delegation through insurance-priced bonds rather than pure capability constraints. The object-capability lineage [6, 5] continues to be relevant: cross-realm operations are exactly a mutual-suspicion scenario in Miller’s vocabulary, and his defensive-consistency framework is the right lens.

Transparency logs. Certificate Transparency [9] is the model; the federated witness log of §2.3 is CT applied to capability tokens. Sigstore [10] and Rekor are the production instantiation we rhyme with most heavily; in-toto [11] and SLSA [12] are the supply-chain analogues that compose with the Federated Harbor for the “poisoned plugin under a bonded principal” case but do not close it. The Crosby–Wallach history-tree [13] is the formal substrate of every modern transparency log including ours.

Cross-domain settlement. Herlihy’s atomic cross-chain swap [15] is a comparison point; the cross-harbor settlement protocol of §6 instead handles non-fungible reputation-priced bonds through conditionally restricted custody. It is not trustless in Herlihy’s sense. Whether trustless settlement is achievable in our setting is open (§12). The IBC protocol [22] supplies another connection/channel comparison; unlike IBC, this design does not assume a consensus-backed state machine underneath.

Agentic-commerce discovery and mandates. The Universal Commerce Protocol [28] (Google/Shopify, 2026) solved cross-administrative discovery for agent-mediated retail with a pattern the Federated Harbor adopts (ADR-0051 Phase 1b): a `/.well-known` JSON profile that does double duty as capability declaration and JWK key publication, with server-selects version negotiation — the party that must execute a request computes the compatibility intersection. Where UCP assumes merchants hold stable HTTPS origins, our daemons often do not (the same constraint that ruled out OpenID Federation’s entity-statement fetching above); the profile therefore also travels as a relay-published mirror. UCP delegates authorization proof to the Agent Payments Protocol’s verifiable-credential mandates [29], whose SD-JWT/JWS/JCS formats the harbor boundary now speaks (ADR-0094). The division of labor is instructive: those protocols standardize discovery and the transaction envelope and are explicit that counterparty trustworthiness, collateral, and settlement enforcement are out of scope — which is the half of the problem this paper and its predecessors supply.

12 Limitations and Open Questions

We close with the honest open frontier. These are not throwaway caveats; they are the questions the paper does not answer, named precisely enough to motivate subsequent work.

12.1 The multi-principal correlated-equilibrium extension

The Bonded Commons paper argued that single-machine coordination is a correlated equilibrium with the daemon as the correlation device [2]. Across two harbors with two daemons, the correlation device fragments. Two candidate resolutions remain unresolved:

- The federated witness log *is* the correlation device, and individual harbors are local enactors. This would extend Bonded’s result intact.
- The cross-harbor case is a partition of the correlated-equilibrium concept into intra-harbor (correlated) and inter-harbor (Nash with shared signal). This would weaken Bonded’s result at the federation layer.

We do not know which is right. The proposal that scoped this paper [3] named Thomas Youle as the load-bearing collaborator for this question (his contribution to Bonded is the closest existing work). The question stays open here.

12.2 Trustless settlement for non-fungible bonds

HTLC-style atomic swaps [15] achieve trustless settlement for fungible assets with a shared hash preimage. The Federated Harbor’s bonds are non-fungible and adjudication-dependent, so that construction does not transfer directly. We adopt conditionally restricted custody (§6) and leave both a trustless construction and an impossibility result open. Either would require a separate threat model and proof.

12.3 Cartel formation across federations

Bonded’s folk-theorem cartel-resistance argument applies inside a single machine. Between machines, cartel formation is structurally easier: colluders can coordinate offline and present a unified front to the witness log, which records that all harbors agreed — a behavior consistent with both honest agreement and collusion. We owe either a strengthening of the bonded mechanism that resists cross-harbor cartels, or a precise statement of the weakening. We have neither.

12.4 Cold-start admission and the invitation-only failure mode

The bonded-sponsor pattern of §7 requires a new harbor to find a sponsor willing to underwrite its probationary risk. In practice, this market may be thin — the historical evidence on whether invitation-bounded federations stay open is mixed [17]. We treat the structural pushback (many sponsors, small probationary scope, witness-log permanence) as engineering discipline, not as theorem.

12.5 Equivocation propagation speed

Property 5.1 separates cryptographic verification from dissemination. The former is constant work once both roots are present; the latter depends on topology, loss, and partitions. Establishing a deployment-specific tail bound is open and is necessary before pricing the witness bond from propagation risk.

12.6 What this means for the reader

A federation paper with five named open questions is doing its job. The point of the paper is to draw the new boundary cleanly so the open problems are visible; if every claim in this paper were closed, there would be no third paper to write. The honest read is that the Federated Harbor is closer to a research agenda than to a deployed product, and the prior two papers (Anchor, Bonded) have a tighter ratio of theorem to conjecture for the same reason. The substrate is real; the federation layer is in progress.

The point of the paper is to draw the new boundary cleanly so that the unsolved problems are visible rather than hidden.

13 Conclusion

The two-machine problem is the structural sequel to the local-swarm problem of Anchor and the trust-as-infrastructure argument of Bonded. The proposed answer is not a new substrate

but a clean extension: keep each daemon sovereign inside its own machine, add a shared witness log that no harbor controls, hang revocation gossip and conditionally restricted custody off the witness log, and gate federation admission on bonded sponsorship rather than permissionless entry. Each primitive does exactly one job. Each primitive is honest about what it does not do.

The proof posture matches the prior papers' discipline. Where we mechanize, we mechanize (with the same ProVerif and TLA+ infrastructure). Where we bound a threat, we name the bound and price the bond against it. Where we leave a question open, we name it. The threat-band diagram of Figure 2 is the most concentrated summary of where each claim lives.

What this paper supplies is a clean federation surface that could compose with the local substrate: a transfer ceremony, witnessed revocation records, a conditional custody design, and bonded admission. The protocol and models make the missing implementation and service-level assumptions explicit; they do not establish that arbitrary harbors can already interoperate safely.

The remaining work—runtime custody enforcement, deployment conformance, a multi-principal equilibrium model, and a cartel-resistance bound—is material. The local substrate is real; the federation described here remains a design with partial mechanization.

References

- [1] Erich Owens. The Anchor Protocol: A Mechanically Analyzed Control Plane for Local Agent Swarms. Curiositech LLC Technical White Paper, May 2026.
- [2] Erich Owens. The Bonded Commons: Pre-Transactional Trust Infrastructure for Multi-Agent Systems. Curiositech LLC Technical White Paper, May 2026.
- [3] Erich Owens. The Federated Harbor: Proposal and Annotated Bibliography. Curiositech LLC working document, May 2026.
- [4] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentczner. Macaroons: Cookies with Contextual Caveats for Decentralized Authorization in the Cloud. In *Network and Distributed System Security Symposium (NDSS)*, 2014.
- [5] Jack B. Dennis and Earl C. Van Horn. Programming Semantics for Multiprogrammed Computations. *Communications of the ACM*, 9(3):143–155, 1966.
- [6] Mark S. Miller. Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control. PhD thesis, Johns Hopkins University, 2006.
- [7] Danny Dolev and Andrew Yao. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983.
- [8] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-Speed High-Security Signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.
- [9] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate Transparency. IETF RFC 6962, June 2013.
- [10] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software Signing for Everybody. In *ACM Conference on Computer and Communications Security (CCS)*, 2022.
- [11] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security Symposium*, 2019.
- [12] Linux Foundation / OpenSSF. SLSA Specification v1.0: Supply-chain Levels for Software Artifacts, 2023.

- [13] Scott A. Crosby and Dan S. Wallach. Efficient Data Structures for Tamper-Evident Logging. In *USENIX Security Symposium*, 2009.
- [14] Alan Demers, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry. Epidemic Algorithms for Replicated Database Maintenance. In *ACM Symposium on Principles of Distributed Computing (PODC)*, 1987.
- [15] Maurice Herlihy. Atomic Cross-Chain Swaps. In *ACM Symposium on Principles of Distributed Computing (PODC)*, 2018.
- [16] John R. Douceur. The Sybil Attack. In *International Workshop on Peer-to-Peer Systems (IPTPS)*, 2002.
- [17] Eve Maler and Drummond Reed. The Venn of Identity: Options and Issues in Federated Identity Management. *IEEE Security & Privacy*, 6(2):16–23, 2008.
- [18] Philip R. Zimmermann. The Official PGP User’s Guide. MIT Press, 1995.
- [19] Alfarez Abdul-Rahman. The PGP Trust Model. *EDI-Forum: The Journal of Electronic Commerce*, 10(3):27–31, 1997.
- [20] UCAN Working Group. UCAN Specification 1.0: User Controlled Authorization Network, 2024.
- [21] Roland Hedberg (ed.), Michael B. Jones, Andreas Åkre Solberg, John Bradley, Giuseppe De Marco, and Vladimir Dzhuvinov. OpenID Federation 1.0. OpenID Foundation Implementer’s Draft 4, July 2024.
- [22] Christopher Goes. The Interblockchain Communication Protocol: An Overview. arXiv:2006.15918, 2020.
- [23] Philipp Schindler, Aljosha Judmayer, Nicholas Stifter, and Edgar Weippl. ETHDKG: Distributed Key Generation with Ethereum Smart Contracts. IACR ePrint 2019/985.
- [24] Andrew Tobin and Drummond Reed. The Inevitable Rise of Self-Sovereign Identity. Sovrin Foundation White Paper, 2017.
- [25] Avery Pennarun et al. How Tailscale Works. Tailscale Engineering Blog, March 2020.
- [26] Juan Brackeva et al. Headscale: Open-Source Tailscale Coordination Server. GitHub project, 2020–present.
- [27] Bruno Blanchet. Modeling and Verifying Security Protocols with the Applied Pi Calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1-2):1–135, 2016.
- [28] Universal Commerce Protocol. Specification and reference implementation (Apache-2.0), Google, Shopify, et al., 2026. <https://ucp.dev>.
- [29] Agent Payments Protocol (AP2): Mandates. Verifiable-credential authorization chain: SD-JWT+kb, JWS detached content, JCS canonicalization, 2026. <https://ap2-protocol.org>.

A Verification Status

Artifact registry. The cross-paper status table in Bonded Commons [2] carries the items that are shared across the series; this appendix lists only the items specific to the Federated Harbor.

Table 1: Federated Harbor verification status. **closed**: model verified and runtime conformance test passes. **PARTIAL**: design specified, mechanization in draft. *open*: known unmechanized; threat band visible in Figure 2 (cobalt).

Claim	Method	Result	Artifact
Single-harbor capability authenticity (inherited)	ProVerif 2.05	closed	Anchor <code>harbor_card_v*.pv</code>
Cross-realm authenticity (§4)	ProVerif (draft)	PARTIAL pending witness-log freshness assumption	<code>fh-xfer.pv</code> (draft)
Attenuation monotonicity across boundary (Lem. 4.1)	ProVerif + Anchor sub-result	PARTIAL	<code>fh-att.pv</code> (draft)
Federated revocation dissemination (Prop. 5.1)	Conditional epidemic model [14]	PARTIAL expected asymptotic result; no partition deadline	<code>fh-conv.tex</code> (proof sketch)
Cross-witness equivocation evidence	Signed-root comparison; dissemination model	PARTIAL verification immediate once views meet; no hard delivery bound	—
Escrow extraction bound (Prop. 6.1)	Transition-system case analysis	PARTIAL conditional on non-bypassable custody; no runtime conformance	<code>fh-escrow.pv</code> (draft)
Cross-harbor Conservation (§9.2)	TLA+ / Apalache, $ F \leq 4$	PARTIAL model-checked at scale	<code>CrossHarborConservation.tla</code>
Trustless settlement for non-fungible bonds	—	<i>open</i> no construction or impossibility proof	—
Multi-principal correlated-equilibrium extension	—	<i>open</i>	—
Cross-federation cartel-resistance bound	—	<i>open</i>	—

What this table means. Every row in the PARTIAL band is committed to and on the active workplan. Every row in the *open* band is a research question we cannot resolve from the desk; the proposal [3] names collaborators whose contributions would be load-bearing. We err toward listing fewer items as closed rather than more, in the same posture as Anchor and Bonded.

B ProVerif Models (draft)

For brevity we omit the full model text in this pre-print; the draft `fh-xfer.pv` extends Anchor’s `harbor_card_v3_delegation.pv` with a second signing key sk_B , an epoch root binding, and a cross-realm authenticity query of the shape sketched in §9.1. The full model will accompany the revised version once the witness-log freshness assumption is mechanized rather than assumed.

C TLA+ Specification (draft)

The full `CrossHarborConservation.tla` extends Bonded’s Conservation specification with per-harbor pools, an escrow pool, and an inter-harbor transit relation. Apalache symbolic model-checking at $|F| = 4$ harbors and bond depth 16 establishes that Conservation holds across all reachable states under the transition relation. The full specification accompanies the revised version.

D Limitations & Boundaries

The formal mechanisms presented in this chapter are mathematically sound within their defined scopes, but they depend on bounding assumptions that must be explicitly acknowledged.

- **Threat Model Exclusions:** Collusion across all independent organs (e.g., the logging daemon, the arbitrator, and the primary execution runtime) is assumed out-of-scope; if all enforcing components are compromised, the guarantees degrade to advisory.
- **Performance Trade-offs:** Cryptographic assertions and WAL-checkpointing for per-write durability incur measurable I/O overhead. These mechanisms are designed for high-stakes coordination, not for bulk data processing or high-frequency trading.
- **Implementation Maturity:** While the core protocols (Anchor, SQLite single-writer) are IMPLEMENTED and running in production, surrounding ecosystem components (like the decentralized judge markets or fully federated multi-sig routing) remain in PARTIAL or DESIGNED phases.

These constraints are not flaws but structural realities of building zero-trust coordination on top of POSIX primitives.